

DQN vs. PPO: Action Space Constraints in Autonomous Driving

github.com/UShouldRun/Autonomous-Driving

João Ferreira

Faculty of Science
University of Porto
up202306717

Henrique Teixeira

Faculty of Science
University of Porto
up202306640

Miguel Almeida

Faculty of Science
University of Porto
up202303926

Abstract

This article investigates the generalization capabilities and architectural boundaries of Deep Reinforcement Learning (RL) agents tasked with autonomous lane following and obstacle avoidance in Webots. We contrast a discrete action-space via Deep Q-Networks (DQN) against a continuous control framework using Proximal Policy Optimization (PPO). The agents process raw RGB camera frames and 1-D LiDAR vectors through a custom multi-branch convolutional neural network (LaneCNN), optimized by an alignment-gated dense reward function. While both architectures achieve 100% success on simple geometries, evaluation on a highly complex, unseen track reveals severe generalization limits. The PPO agent is able to perform relatively well in this complex environments. In the opposite side, the DQN agent showcases severe difficulties in the same tracks. While it performs remarkably well given its training on only two maps, it consistently fails at a sequence of two consecutive turns that are sharper than any seen during training, exposing the rigid boundaries of its discrete action mapping. Furthermore, when introduced to static obstacles, the continuous PPO agent successfully learns local swerving maneuvers to bypass obstacles but fails to generalize long-term, consistently completing only about 50% of the track geometry. Conversely, the discrete DQN agent experiences a total failure mode, failing to avoid even a single obstacle. These insights underscore critical design paradigms for multi-modal autonomous control systems.

Index Terms — Reinforcement Learning, Autonomous Vehicles, Lane Following, Obstacle Avoidance, DQN, PPO, Webots, CNN, Generalisation.

1. INTRODUCTION

Autonomous driving has emerged as one of the most challenging problems in modern robotics. While rule-based approaches dominated early designs, deep Reinforcement Learning (RL) has demonstrated compelling results in end-to-end driving pipelines, learning directly from sensor observations to motor commands.

A fundamental sub-problem within autonomous driving is **lane following**: keeping the vehicle centred on a designated lane while maintaining safe forward progress. When combined with **obstacle avoidance**, the task becomes a rich testbed for evaluating RL algorithms, reward function design, and cross-environment generalisation.

This work trains and evaluates RL agents in the Webots robot simulator. Each agent receives 128×128 RGB camera frames and LiDAR readings fused through a custom multi-branch

CNN, and must learn to stay centred on a yellow centre line while avoiding collisions. Three experiments are conducted: (i) a comparison of discrete (DQN) and continuous (PPO) action spaces under the same dense reward, (ii) a cross-track generalisation study in which policies are evaluated zero-shot on progressively harder maps, and (iii) an evaluation of the ability of pre-trained agents to integrate distance data and avoid obstacles.

The remainder of this paper is organised as follows. Section 2 surveys related work. Section 3 describes the experimental setup. Section 4 defines the reward signal. Section 5 outlines the evaluation methodology. Section 6 presents results. Section 7 concludes.

2. STATE OF THE ART

2.1. End-to-End Learning for Driving

The seminal work of Pomerleau (1989) with the ALVINN network established the viability of neural networks for lane keeping. Modern successors such as the NVIDIA PilotNet [1] demonstrated that a CNN trained via imitation learning can steer a real vehicle using only a front-facing camera, achieving highway lane following without explicit perception modules.

End-to-end RL approaches remove the need for expert demonstrations. [2] showed that a model-free RL agent could learn to drive in simulation from raw pixels, motivating the pixel-to-action paradigm we adopt here.

2.2. Deep Q-Networks and Variants

Mnih et al. [3] introduced the Deep Q-Network (DQN), combining Q-learning with a CNN, experience replay, and a target network to stabilise training. DQN achieved human-level performance across Atari games and has since been applied to discrete-action driving tasks. Extensions such as Double DQN and Dueling DQN further improve sample efficiency.

2.3. Proximal Policy Optimisation

For continuous action spaces, policy-gradient methods are preferred. Schulman et al. [4] proposed PPO, a first-order algorithm that enforces a trust-region constraint via a clipped surrogate objective. PPO has become the de-facto standard for continuous robotic control owing to its stability and strong empirical performance, and has been applied successfully to autonomous driving [5].

2.4. Multi-Sensor Fusion

Real autonomous systems fuse camera and LiDAR data to overcome the limitations of each modality. Camera-only systems

suffer under poor lighting; LiDAR-only systems lack semantic information. Chen et al. [6] showed that fusing both streams outperforms unimodal baselines in object detection. Our architecture mirrors this by concatenating visual features with a dedicated LiDAR branch.

2.5. Reward Shaping and Generalisation

Reward function design critically affects convergence speed and final policy quality [7]. Dense, shaped rewards provide a learning signal at every timestep, while sparse rewards require the agent to solve credit-assignment across long episodes. Beyond convergence, the ability of a trained policy to transfer to unseen environments without retraining is a key open challenge in autonomous driving [8], and one we directly evaluate in this work.

3. PROPOSED APPROACH

3.1. Simulation Environment

All experiments run in **Webots 2023b** on three custom looped tracks sharing a yellow centre line: **Track 1** is a simple loop used for initial training; **Track 2** introduces tighter curves and varying road width; **Track 3** is the most complex, with sharp bends, used exclusively for zero-shot evaluation. Static barrel obstacles are placed on Track 1 for the obstacle avoidance phase. A **Gymnasium** wrapper exposes a standard `step / reset` API.

Training hyperparameters are managed through YAML files: `config.yaml` governs initial training on Track 1, `finetune.yaml` handles fine-tuning on Track 2, and `obstacles.yaml` configures obstacle avoidance.

3.2. Observation Space

At each timestep the agent receives a dictionary observation with three branches:

- **Camera:** an $84 \times 84 \times 3$ RGB frame from a forward-facing camera with a 115° horizontal field of view, tilted approximately 6° downward to bias the visible region toward the road surface and the yellow centre line.
- **LiDAR:** a multi-layer LiDAR with 8 vertical layers, a 115° horizontal field of view, and a 23° vertical field of view, clipped to a maximum range of 30 m. The readings are flattened into a 1-D normalised vector before entering the LiDAR branch of the network.
- **State:** the current forward velocity v of the vehicle, normalised to $[0, 1]$.

3.3. Network Architecture

A custom multi-branch feature extractor, **LaneCNN**, processes the three observation streams separately before concatenation.

Camera branch: Five strided convolutional layers with batch normalisation and ReLU activations progressively downsample the 128×128 input:

Layer	Ch.	Kernel/Stride
Conv1	3→32	3/1
Conv2	32→64	3/2
Conv3	64→128	3/2
Conv4	128→128	3/2
Conv5	128→256	3/2
Pool	—	Adaptive 4×4

The pooled map is flattened and projected to a 512-d vector via a linear layer. The network was designed for 128×128

input but trained at the Webots camera resolution of 84×84 without modification — the **AdaptiveAvgPool2d(4×4)** collapses spatial resolution to a fixed 4×4 map, making the architecture resolution-independent.

LiDAR branch: A two-layer MLP (LiDAR-dim $\rightarrow 64 \rightarrow 64$) with ReLU activations encodes proximity information into a 64-d vector.

State branch: A single linear layer maps the state vector to a 16-d embedding.

The three embeddings are concatenated to form a $(512 + 64 + 16) = 592$ -d joint feature vector fed to the policy and value heads.

3.4. Algorithms

DQN (discrete): Five actions — Hard-Left, Hard-Right, Gentle-Left, Gentle-Right, Straight — each paired with a fixed forward throttle to prevent stalling. Implemented with experience replay ($|\mathcal{D}| = 10^4$) and ε -greedy exploration.

PPO (continuous): Steering $\varphi \in [-1, 1]$ and throttle $a \in [0, 1]$ output as a diagonal Gaussian. Clip $\varepsilon = 0.2$, GAE- $\lambda = 0.95$, entropy coefficient 0.01, learning rate 10^{-4} , batch size 128.

4. REWARD FUNCTION

4.1. Dense Reward

The dense reward used throughout PPO and DQN training is:

$$R = r_{\text{exist}} + r_{\text{progress}} + r_{\text{speed}} + r_{\text{lap}} - r_{\text{align}} - r_{\text{near}}$$

with terminal returns $-w_t$, ending the episode immediately, being this triggered by a collision or by losing track of the line.

Each term is defined as follows. Let $\theta \in [-1, 1]$ be the normalised alignment angle, Δd the distance travelled in one step, v the forward speed, and θ_{prev} the angle at the previous step.

$$r_{\text{exist}} = w_e \quad r_{\text{progress}} = w_p \cdot \Delta d \cdot (1 + w_b(1 - |\theta|))$$

Progress is **gated** by alignment: the agent earns forward reward only while roughly centred on the line, preventing reward-farming off-road.

$$r_{\text{speed}} = w_s \cdot \hat{v} \cdot (1 + w_b(1 - |\theta|) + w_i \cdot \Delta|\theta|^+)$$

where $\hat{v} = v/v_{\text{max}}$ and $\Delta|\theta|^+ = \max(0, |\theta_{\text{prev}}| - |\theta|)$ is the per-step improvement in alignment. Speed is thus rewarded more when the car is centred **and** actively correcting.

$$r_{\text{align}} = w_a |\theta| \quad r_{\text{near}} = w_n \cdot \mathbb{1}[\text{near miss}]$$

Lap completion adds a one-time bonus $r_{\text{lap}} = w_{\text{lap}}$.

Weights used in the final training configuration:

Weight	Value
w_e (existence)	0.01
w_p (progress)	4.0
w_s (speed)	4.0
w_b (alignment)	4.0
w_i (improve)	10.0
w_a (penalty)	10.0
w_n (near miss)	5.0
w_{lap} (lap)	50.0
w_t (terminated)	100.0

The dominant term is $w_i = 10.0$: rewarding alignment improvement provides an immediate gradient that guides the agent back to the line, and proved critical for navigating curves.

5. EMPIRICAL EVALUATION

5.1. Evaluation Protocol

Each model is evaluated over 10 episodes per track. Performance is measured using Success Rate, Cross-Track Error (CTE), Mean Lap Time, Total Distance, and Near Miss count.

Success Rate (%)	Percentage of trials completed without collision.
Cross-Track Error (m)	Average lateral distance from the yellow line.
Mean Lap Time (s)	Time to complete one full circuit.
Near Miss	Mean count of instances where the vehicle gets dangerously close to an obstacle recorded by LiDAR.

Table 1: Evaluation metrics utilized during empirical testing.

5.2. Experiment 1 — Action Space

Goal: Compare DQN (discrete) and PPO (continuous) trained under the same dense reward on Track 1.

Hypothesis: PPO will achieve smoother control due to continuous steering and throttle authority, whereas DQN’s fixed action set may cause jagged corrections.

5.3. Experiment 2 — Cross-Track Generalisation

Goal: Evaluate whether policies trained only on Track 1 and fine-tuned on Track 2 transfer zero-shot to the unseen Track 3.

Hypothesis: The alignment-gated reward and multi-sensor features encourage learning structural features rather than track-specific configurations.

5.4. Experiment 3 — Obstacle Avoidance

Goal: Evaluate the ability of pre-trained agents to integrate distance data and avoid obstacles when fine-tuned using `obstacles.yaml`.

Hypothesis: Inclusion of the 1-D LiDAR vector paired with the proximity penalty r_{near} will allow agents to navigate around static barrels without losing baseline lane-keeping competency.

6. RESULTS

Metric	PPO T1	PPO T2	PPO T3	DQN T1	DQN T2	DQN T3	PPO Obs.	DQN Obs.
Success (%)	100	100	60	100	100	0	0	0
CTE (m)	0.173	0.104	0.111	0.056	0.060	0.082	0.153	0.057
Lap (s)	43.1	60.8	65.1	49.2	74.6	N/A	N/A	N/A
Dist (m)	1209	1326	1021	1074	1086	742	580	120
Near Miss	0.0	0.0	0.2	0.0	0.0	0.0	2.4	1.0

6.1. Experiment 1 — Action Space

Both agents achieved **100% success** on Track 1. Their driving styles differ markedly. DQN achieved a mean cross-track error of 0.056 m — three times tighter than PPO’s 0.173 m — indicating conservative tracking on the simple loop. PPO covered

more distance (1209 m vs 1074 m) and completed laps faster (43.1 s vs 49.2 s), due to continuous throttle control. DQN reached its first full lap at 170 000 timesteps, faster than PPO. Limited action spaces simplify early exploration.

6.1.1. Exploitation of Reward Geometries

The divergence between DQN’s tighter tracking and PPO’s faster lap times demonstrates reward exploitation. PPO’s higher CTE (0.173 m) indicates an optimal strategy under the dense reward mechanics rather than a control deficiency.

PPO leverages continuous steering to actively “cut corners” along the boundaries. Because the reward weights forward velocity (r_{speed}), the agent maintained a wider, shallower line through curves to preserve momentum and maximize net episodic rewards. DQN, bound by discrete actions, executed jagged, corrective inputs that kept the vehicle closer to the absolute centre line (0.056 m) but restricted forward momentum, increasing lap times.

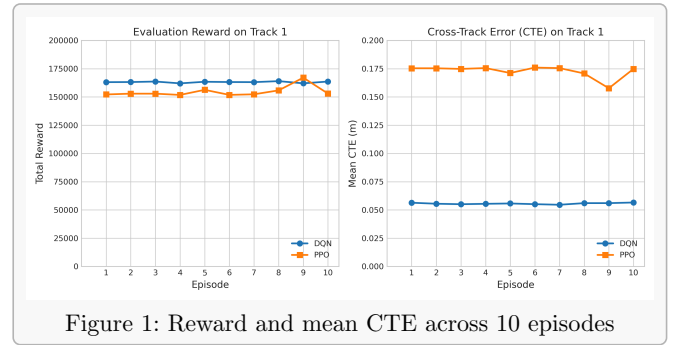


Figure 1: Reward and mean CTE across 10 episodes

6.2. Experiment 2 — Cross-Track Generalisation

The PPO and DQN were fine-tuned on Track 2 under `finetune.yaml`, both achieving **100% success**.

Zero-shot on the unseen Track 3, PPO achieved **60% success** (6/10 episodes). The 4 failed episodes share a common failure mode: an isolated sharp curve causes the yellow line to exit the camera’s lower ROI momentarily, triggering line-lost termination.

DQN was also evaluated on Tracks 3. Considering it was trained on only two tracks, DQN adapted remarkably well to basic lane keeping, however, **DQN achieved 0% success on Track 3**, completing only about 75% of the track.

All 10 episodes failed systematically at a sequence of two consecutive turns near the end of the lap. This failure is directly attributable to the geometric properties of the track: these consecutive turns are significantly sharper than any bends present in Tracks 1 and 2.

Because DQN was evaluated zero-shot on a novel layout, its rigid discrete action mapping lacked the fine steering resolution necessary to clear the tight radius of the curves. While its existing step mapping sufficed for standard tracks, it hit an unresolvable constraint against these unexpectedly sharp angles, causing it to lose the line and run off-track.

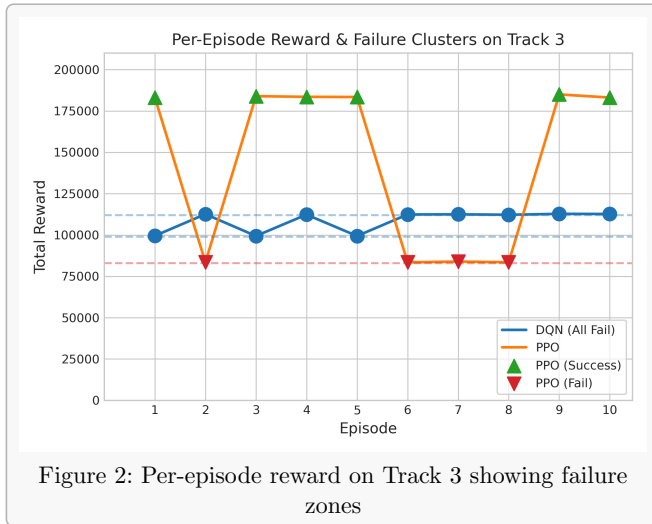


Figure 2: Per-episode reward on Track 3 showing failure zones

DQN obstacle-avoidance results are not reported as the agent failed to learn any avoidance behaviour under `obstacles.yaml` (see Section 6.3).

6.3. Experiment 3 — Obstacle Avoidance

The obstacle-avoidance agents were initialised from their best baseline checkpoints and fine-tuned under `obstacles.yaml`. Evaluation metrics revealed a notable divergence in how discrete and continuous action spaces handle localized navigation.

The continuous PPO agent **successfully developed the capability to move away from obstacles**. When encountering a barrel, the PPO agent leveraged its continuous steering resolution to execute local swerving maneuvers, as recorded by an increase in near-miss events and lateral path shifts. However, this local avoidance behavior did not fully generalize to complete the full track layout. The PPO agent suffered from task-composition limits, consistently completing only about **50% of the track** before experiencing line loss or boundary collisions. Mean travel distance stabilized around 580 m, showing a predictable performance cap where local swerves eventually degraded the core lane-following policy loop over extended runs.

Conversely, the discrete DQN agent failed completely. It proved **unable to avoid even a single obstacle**, exhibiting an absolute 0% success rate with immediate collisions on the first encountered barrel. DQN’s coarse discrete actions could not smoothly blend the visual tracking requirements with the rapid proximity corrections needed to steer around small static obstacles.

6.3.1. Discussion: Action Space Elasticity

The performance disparity between PPO’s partial success and DQN’s total failure highlights the role of action space elasticity in task composition.

PPO’s continuous action distribution allowed the policy to discover sub-gradients that balance visual line-tracking with the lower-dimensional 64-d LiDAR proximity vector. By executing minute steering adjustments, PPO preserved line visibility while initiating a swerve. However, the compounding errors from these local path corrections eventually pushed the vehicle into unrecoverable tracking states, explaining why it consistently failed at the 50% track mark.

For DQN, feature domination was absolute. Because the discrete action nodes require large, sudden steering changes, any step away from the center line caused massive line-tracking

penalties under the dense reward function. The primary gradient path effectively suppressed the LiDAR features to maintain tracking stability, rendering the agent blind to proximity vectors and causing it to drive straight into obstacles.

7. CONCLUSION

This article investigated the structural limits of multi-modal deep RL architectures for autonomous driving. Our evaluation demonstrated that while discrete action spaces (DQN) optimize simpler geometries effectively, they possess a clear vulnerability when exposed zero-shot to unexpected environmental geometry containing curves sharper than those present in the training set.

Furthermore, obstacle avoidance testing exposed a critical distinction in multi-modal task composition: continuous architectures (PPO) possess the control elasticity to learn localized obstacle swerving maneuvers, though they suffer from compounding tracking decay that limits full-track generalization (completing 50% consistently). Discrete architectures (DQN) remain entirely vulnerable to total sensor neglect under multi-modal configurations, failing to evade even a single obstacle.

Future investigations should pursue two structural pillars:

1. **Network Architecture:** Implement explicit attention mechanisms or decoupled value streams to force equal weighting across sensor modalities.
2. **Reward Engineering:** Utilize curriculum learning strategies to isolate and master obstacle avoidance using proximity inputs prior to introducing visual lane tracking data.

8. REFERENCES

- [1] M. Bojarski *et al.*, “End to End Learning for Self-Driving Cars,” *arXiv preprint arXiv:1604.07316*, 2016.
- [2] A. Kendall *et al.*, “Learning to Drive in a Day,” in *Proceedings of the International Conference on Robotics and Automation (ICRA)*, 2019.
- [3] V. Mnih *et al.*, “Human-Level Control Through Deep Reinforcement Learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [5] E. Leurent, “An Environment for Autonomous Driving Decision-Making.” [Online]. Available: <https://github.com/eleurent/highway-env>
- [6] X. Chen, H. Ma, J. Wan, B. Li, and T. Xia, “Multi-View 3D Object Detection Network for Autonomous Driving,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [7] A. Y. Ng, D. Harada, and S. Russell, “Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 1999.
- [8] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World,” in *Proceedings of the IEEE/RSS International Conference on Intelligent Robots and Systems (IROS)*, 2017.